Highlights

How close do WSN routers come to the lifetime bound? A reproducible benchmark of shortest-path, coordinated and learned multi-hop routing

Shubham Mittal, Aditya Periwal

- Multi-hop WSN routers are scored against a fractional-flow lifetime bound.
- Battery-weighted Dijkstra reaches 85% of the bound with no training.
- Within-round coordination adds about 4.5 points; two mechanisms tie.
- An LP planner reaches 97%, certifying the bound is nearly tight.
- A corrected learned-router reconstruction collapses; we diagnose why.

How close do WSN routers come to the lifetime bound? A reproducible benchmark of shortest-path, coordinated and learned multi-hop routing

Shubham Mittal[a], Aditya Periwal[a,*]

[a]*Netaji Subhas University of Technology, New Delhi, 110078, India*

Abstract

Learning-based routers for wireless sensor networks are rarely compared with strong shortest-path baselines or with a lifetime bound, so their benefit is hard to judge. We benchmark multi-hop tree routing without aggregation against the Chang–Tassiulas fractional-flow bound, extended to an oracle for time-varying traffic, with every control bit charged: battery reports and route advertisements that encode every next hop. On 30 topologies under four traffic scenarios, battery-weighted Dijkstra reaches 85.2% of the bound. Coordinating the routes chosen within a round adds about 4.5 points. Chang and Tassiulas's flow augmentation, run in within-round increments, reaches 89.9%, and a sequential booked-load Dijkstra 89.7%; at this operating point the two are indistinguishable (difference -0.2 points, 95% CI $[-1.3, 0.8]$). An LP planner re-solved every round reaches 96.6%, which shows that the bound is on average within 3.6% of what a sequence of trees can achieve. On fresh topologies neither coordination mechanism dominates. Flow augmentation is ahead in small and clustered networks. Booked-load Dijkstra is ahead where traffic funnels hardest: with the sink at a corner it reaches 85.6% against 79.3%. Under lossy links with retransmissions and contention, coordination keeps its lifetime gain but lowers the delivery ratio. A reconstruction of a recent multi-agent reinforcement-learning router, with its reward and selection protocol corrected, collapses to direct transmission, although its own reward prefers Dijkstra routes eightfold; we report this as a reproducibility failure. When the sink can afford an LP per round, the planner is usually the better

*Corresponding author.

Email address: [corresponding author e-mail] (Aditya Periwal)

choice.

Keywords: Wireless sensor networks, Network lifetime, Energy-efficient routing, Benchmarking, Maximum-lifetime linear program, Flow augmentation, Reproducibility

---

1. Introduction

A wireless sensor network (WSN) forwards readings from battery-powered nodes to a sink over several hops. Every relayed bit costs energy, so the routing decides how long the network lasts. The time until the first node dies is a standard lifetime measure [1].

Recent work proposes deep reinforcement learning for this routing problem [2, 3]. These methods are usually compared with clustering heuristics, metaheuristics or other learned methods. They are rarely compared with a battery-weighted shortest-path router, and rarely reported as a fraction of an achievable bound (Table 1). Without both, it is hard to tell how much a new method contributes.

This paper is primarily a benchmark. We score routers against the Chang–Tassiulas fractional-flow bound [1], extended to an oracle that knows future traffic, on held-out and fresh topologies. Every control bit the routers rely on is charged. We restrict the comparison to one family, multi-hop trees without aggregation, so that no protocol is evaluated outside the setting it was designed for. Within that family the results fall into clear levels:

- **Battery-weighted Dijkstra** reaches about 85% of the bound, with no training.
- **Within-round coordination** adds about 4.5 points. Here the routes of one round account for each other, either through Chang and Tassiulas's flow augmentation run in small increments or through a sequential booked-load Dijkstra. The two mechanisms tie at the main operating point but not elsewhere: on fresh topologies flow augmentation is ahead in small and clustered networks, and booked-load Dijkstra where traffic funnels hardest towards the sink.
- **An LP planner** re-solved every round reaches about 97%. This shows that the relaxation is close to what trees can achieve, and that the remaining gap belongs to the solver.

Contributions.

(C1) **A bound-normalised benchmark protocol.** It has charged control accounting, same-family baselines tuned on separate validation topologies, the topology as the statistical unit with bootstrap intervals, robustness on fresh topologies, and a released artifact.

(C2) **A measured ranking with a tightness certificate.** At the main operating point the best realised router reaches 96.4% of the bound on average, so the bound overstates the achievable lifetime by at most a few percent there and normalised scores can be read close to absolute. Under a corner sink with clustered nodes the certificate is much weaker (Section 6.5).

(C3) **Within-round coordination, in two complementary forms.** Flow augmentation (2004) and booked-load Dijkstra each add about 4.5 points over battery Dijkstra at the main operating point, where they tie. On fresh topologies they diverge: booked load gains up to 26 points over flow augmentation when the sink is at a corner of a clustered field, and flow augmentation gains about 4 points in small or clustered networks. Removing the battery term destroys either. Reporting load in quantised tiers costs nothing measurable, and it is unnecessary when the sink observes the traffic.

(C4) **Realism and cost probes.** We report lossy links with ARQ and receiver contention, and the measured sink computation time from $n = 50$ to $n = 800$, which shows when avoiding the LP solver matters.

(C5) **A reproducibility record.** We document the errors found in earlier versions of this study and corrected here (Appendix Appendix A). We also report a learned-router reconstruction that fails, with a diagnosis (Appendix Appendix B).

Section 2 reviews related work. Section 3 defines the model, its scope and its assumptions, Section 4 the routers and Section 5 the protocol. Section 6 reports the results, Section 7 discusses them together with the threats to validity, and Section 8 concludes.

2. Related work

2.1. Energy models and clustering

Heinzelman et al. [4] introduced the first-order radio model used by most WSN routing studies, including this one (Section 3.2). Their LEACH protocol popularised clustering with rotating cluster heads. Later schemes elect cluster heads with fuzzy c-means or population metaheuristics. These protocols derive much of their benefit from in-network aggregation at cluster heads. This paper studies multi-hop tree routing *without* aggregation, which is MADII's own setting [2]. Cluster-based protocols are therefore outside its scope and are not used as comparators (Section 3.1).

2.2. Lifetime-optimal and energy-aware shortest-path routing

Dijkstra's algorithm [5] with radio energy as the link cost gives minimum-total-energy (MTE) routing, which exhausts the same relays first. Chang and Tassiulas [1] formulated maximum-lifetime routing as a linear program (LP) over link flows. They also proposed *flow augmentation*: shortest-cost paths whose link costs combine energy per bit with the residual energy at both ends, recomputed as flow is added in small increments. They reported lifetimes close to the LP optimum and compared against MTE routing. Flow augmentation already coordinates successive routing decisions through the predicted residual energy, so it is the natural solver-free baseline for this study, and we evaluate it (Section 4.3). Faster single-source shortest-path algorithms [6] compute the same paths and cannot change routing quality.

2.3. Load-balanced routing trees

Dai and Han [7] grow a tree from the sink by grafting heavy nodes onto light branches; as described in [8], their method ignores distance and energy. Shan et al. [8] build a load-balanced *shortest-path* tree, in which every sensor reaches the sink in its minimum hop count, by a layer-by-layer network-flow construction. With non-aggregated relaying they report about 85% of an upper bound, and we implement their construction as a baseline. EBR-DA [9] combines residual energy and buffer occupancy over a hop tree and is designed for an aggregation setting. Because our model has no buffers and no aggregation, we do not use it as a comparator. Combining energy and load in a route decision is therefore established, and so is building load-balanced trees.

Table 1: Evaluation elements reported by the closest studies, as stated in each paper. ✓: reported; ∼: partly; ✗: not reported.

Study	Shortest-path baseline	Lifetime bound	Time-varying traffic	Control energy charged	Statistical test	Code released
Chang and Tassiulas [1]	✓	✓	✓	✗	✗	✗
Shan et al. [8]	✗	✓	✗	✗	✗	✗
Mahdi et al., EBR-DA [9]	✗	✗	∼	✗	✗	✗
Younus et al. [3]	✗	✗	✗	✓	✗	✗
Yang et al., MADII [2]	✗	✗	✗	∼	✗	✗
This study	✓	✓	✓	✓	✓	✓

Notes. Shan et al. compare with a node-centric tree and an upper bound. EBR-DA is evaluated on event-driven traffic against DRINA and InFRA. Younus et al. measure energy on a testbed, so control traffic is included physically. MADII charges a 0.1-kbit control packet but not the encoding of next-hop identifiers.

2.4. Learning-based routing

MADII [2] is a multi-agent deep Q-network with an Informer encoder [10]. Under centralised training and execution it selects every sensor's next hop. It is compared with clustering, metaheuristic and learned methods, but not with a shortest-path baseline or a lifetime bound. Younus et al. [3] train an RL agent in a software-defined WSN controller on a testbed and compare it with an RL-based WSN routing scheme. Tang et al. [11] apply multi-agent RL to topology optimisation in wireless mesh networks, targeting delay and power. A recent review [12] covers learning for routing more broadly, and recent comparison studies tabulate results from different datasets and units [13]. In network traffic classification, careful baselines have matched deep models [14]; this is a different task, but it is the same evaluation concern.

2.5. Evaluation practice

Table 1 records, for the closest studies, which evaluation elements each paper reports, as stated in its own text. Every criterion can be checked against the cited paper.

3. System model, scope and assumptions

3.1. Network model and scope

There are n static sensor nodes $V = \{1, \ldots, n\}$ and one sink s. The main setting is MADII's published setting [2]: $n = 100$ nodes uniform in a 500×500 m field, with the sink at the centre and initial energy $E_0 = 0.5$ J. Time is divided into rounds. In round t node i generates $k_i(t)$ packets, each forwarded to the sink along the round's routing tree $p_t : V \to V \cup \{s\}$. Relays forward every bit they receive, as in MADII ("no data compression").
Scope. The study concerns multi-hop tree routing without in-network aggregation. Protocols whose benefit comes mainly from aggregation at cluster heads (LEACH-family clustering) or that are specified for an aggregation setting (EBR-DA) are therefore not used as comparators. Comparing them with full forwarding would test them outside the setting they were designed for.

3.2. Radio energy model

Transmitting k bits over distance d and receiving k bits cost

$$E_{\mathrm{TX}}(k, d) = \begin{cases} kE_{\mathrm{elec}} + k\,\varepsilon_{\mathrm{fs}} d^2, & d < d_0, \\ kE_{\mathrm{elec}} + k\,\varepsilon_{\mathrm{amp}} d^4, & d \geq d_0, \end{cases} \tag{1}$$

$$E_{\mathrm{RX}}(k) = kE_{\mathrm{elec}}, \qquad d_0 = \sqrt{\varepsilon_{\mathrm{fs}}/\varepsilon_{\mathrm{amp}}} \approx 87.7 \text{ m}, \tag{2}$$

with $E_{\mathrm{elec}} = 50$ nJ/bit, $\varepsilon_{\mathrm{fs}} = 10$ pJ/bit/m^2 and $\varepsilon_{\mathrm{amp}} = 0.0013$ pJ/bit/m^4. We write $e_{\mathrm{tx}}(i, j) = E_{\mathrm{TX}}(1, d_{ij})$ and $e_{\mathrm{rx}} = E_{\mathrm{elec}}$.

3.3. Packet layer and control accounting

Every bit any router relies on is charged, and the same charges apply to every router (Table 2):

(i) **Data packets** carry 4,000 payload bits plus a 16-bit residual-battery report from the source, so $L = 4{,}016$ bits travel on every hop. Every energy-aware router, the LP planner and MADII's state upload need the batteries at the sink. The report is therefore charged to all routers, not assumed free.

Table 2: Network, radio and packet parameters.

Parameter	Value
Nodes, field, sink	100, uniform in 500×500 m, centre
Initial energy E_0	0.5 J (trace bench 1.0 and 1.5 J)
Data packet L	4,000 payload + 16 battery bits
Advertisement L_c	$100 + n\lceil \log_2(n+1) \rceil$ (800 at $n = 100$)
Radio constants	E_{elec} 50 nJ/bit; ε_{fs} 10 pJ/bit/m^2; ε_{amp} 0.0013 pJ/bit/m^4
Aggregation	none (full forwarding)
Tiered-load ablation	+8 bits per packet; $+2n$ bits per advertisement

(ii) **The route advertisement** must encode every node's next hop. It has a 100-bit header plus $n\lceil \log_2(n+1) \rceil$ bits, so $L_c = 800$ bits at $n = 100$, and every alive node pays $E_{\text{RX}}(L_c)$ per round. An earlier version of this study charged only 100 bits, which cannot encode 100 identifiers; Appendix Appendix A lists this and the other corrections.

(iii) **Traffic rates** used by the routers are counted by the sink from the packets it receives, which costs no extra bits. The tiered-load variant (Section 4.5) instead reports load in 2-bit tiers on an extra byte per packet plus a $2n$-bit snapshot in the advertisement. It is charged accordingly and is an ablation only.

A relay that cannot afford a transmission dies within the round, and the packets routed through it in that round are lost. A source whose path failed retries directly to the sink if its battery allows. **Analytical, round-based model.** Like MADII [2], whose results are computed from the same radio equations, the bench is analytical rather than a packet-level discrete-event simulation. Each round, every node's energy is computed in closed form from the bits it sends and receives under (1)–(2). We keep this model deliberately, so that every router, including our reconstruction of MADII, is compared on MADII's own terms and against a bound expressed in the same equations. Consequently no MAC frames, acknowledgements or retransmissions appear in the main bench. Section 6.6 extends the same equation-based model with lossy links, ARQ and contention as a probe.

3.4. Traffic

A day is 24 rounds, and every router receives 3 days of pre-deployment history. In the daily-surge scenario (R), the 20% of nodes nearest a random point send Poisson traffic with mean 4 packets per round during a 6-round evening and 0.5 otherwise, and all other nodes have mean 1. In the constant scenario (N), every node sends one packet per round, which is MADII's static setting. Bursts (B) adds to R, each round with probability 0.05, a burst in which a random node sends five times its rate for 4 rounds. Pattern shift (D) moves the hotspot to a new set of nodes after two days; it is a robustness scenario that nothing is tuned on. As a second, trace-driven traffic generator we convert Intel Berkeley Lab readings [15] into packets by send-on-delta reporting (one heartbeat per hour plus a packet per 0.5 °C or 100 lux change) on the 54-mote layout scaled to the 500 m field. This yields traffic with a different autocorrelation structure. It is not measured radio traffic.

3.5. Assumptions

A1 Static nodes with identical initial energy; no harvesting or battery replacement.

A2 One static, mains-powered sink; its computation is not charged to sensors (its cost is reported separately in Section 6.8).

A3 No obstacles; link cost depends only on distance through (1)–(2).

A4 Any node can reach any other node and the sink by power control.

A5 Main bench: ideal symmetric links and collision-free scheduling. Section 6.6 relaxes this.

A6 Only transmission and reception consume energy; sensing, processing and idle listening are not modelled.

A7 No aggregation or compression.

A8 Round-based operation: the sink computes each round's tree before the round and broadcasts it in the advertisement.

A9 The sink knows every node's position.

A10 The sink learns residual batteries from the 16-bit reports of the previous round (charged, item (i)).

A11 The sink learns traffic only from packets it has received; only the oracle bound sees future traffic.

A12 Nodes fail only by depletion; a relay that cannot afford its transmission dies within the round.

A13 Every router sees the same deployments and traffic, so all comparisons are paired.

3.6. Lifetime, the bound and its tightness

Let F be the round in which the first node dies; $T = F - 1$ completed death-free rounds is the lifetime. For example, if the first death happens during round 152, then $F = 152$ and $T = 151$. Let $f_{ij} \geq 0$ be the total bits sent on link $i \to j$. For constant traffic, the fractional-flow ceiling [1] is

$$\max_{T,\, f \geq 0} T \quad \text{subject to, for every } i \in V,$$

$$\textstyle\sum_j f_{ij} - \sum_k f_{ki} = L\,T, \tag{3}$$

$$\textstyle\sum_j f_{ij}\, e_{\mathrm{tx}}(i,j) + \sum_k f_{ki}\, e_{\mathrm{rx}} + T\, E_{\mathrm{RX}}(L_c) \leq E_0. \tag{4}$$

For time-varying traffic the right-hand side of (3) becomes $L \sum_{t<T} k_i(t)$, and T^* is the largest T for which the LP is feasible, found by bisection (an oracle that knows the future). Any sequence of routing trees that keeps all nodes alive for T rounds gives a feasible flow, so $T \leq T^*$. We report every router as $100\,T/T^*$. This is a percentage of a *relaxation*, not of the best achievable sequence of trees. Its looseness is bounded empirically in Section 6.3: on every case, the best realised router certifies that the achievable optimum lies between that router's lifetime and T^*.

4. Routers

Every router computes the round's tree at the sink from the information in A8–A11 and broadcasts it in the advertisement. All routers in the main comparison build multi-hop trees with full forwarding.

4.1. Shortest-path baselines

Min-energy (MTE) Dijkstra uses the link cost $e_{\mathrm{tx}}(i,j) + e_{\mathrm{rx}}$. *Battery-weighted Dijkstra* multiplies it by E_0/E_i, the sender's inverse residual-battery fraction, and is recomputed every round:

$$w(i,j) = \left[e_{\mathrm{tx}}(i,j) + e_{\mathrm{rx}}\right] E_0/E_i. \tag{5}$$

In battery-weighted Dijkstra every node's path is computed independently, so within one round many nodes can select the same relay.

4.2. Load-balanced shortest-path tree (Shan et al.)

We implement the top-down construction of [8]. Hop layers are built over a radio range R. Each new layer is attached to the previous one so that the largest branch load at the sink is minimised, using a bisection on the bound with a max-flow feasibility test and unit load per sensor, as in their model. Within the chosen branch a node takes the candidate parent with the fewest descendants. The tree is rebuilt when a node dies. We do not implement their distributed refinement step. R is tuned on validation deployments ($R = 110$ m).

4.3. Flow augmentation (Chang and Tassiulas)

Flow augmentation [1] routes traffic in small increments on shortest-cost paths, updating the residual energy after each increment. We run it as a per-round planner. Over a horizon of H rounds, each source's traffic is pushed in K increments on shortest paths over the cost

$$c(i,j) = e_{\mathrm{tx}}(i,j)\,(E_0/E'_i)^x + e_{\mathrm{rx}}\,(E_0/E'_j)^x, \tag{6}$$

where E' is the residual energy predicted after the increments already placed. Each node then forwards to the next hop that carries the largest share of its augmented flow. No solver is used. Validation chose $x = 1$, $H = 3$ and $K = 30$ (Section 5). Because E' changes during the round's own increments, later increments avoid relays loaded by earlier ones. This is a form of within-round coordination.

4.4. Booked-load Dijkstra

Booked-load Dijkstra coordinates one round's routes through *load* instead of predicted energy. The sink processes nodes in increasing distance from the sink. For each node without a route it runs Dijkstra over

$$w(j,k) = \big[e_{\mathrm{tx}}(j,k) + e_{\mathrm{rx}}\big]\,\frac{E_0}{E_j}\Big(1 + \kappa\,\frac{A_j}{\bar{r}}\Big), \tag{7}$$

where A_j is the traffic already booked through node j in this round's construction and $\bar{r}$ is the mean node rate. It then fixes the path and books the

Algorithm 1 Booked-load Dijkstra: one round at the sink

Require: alive nodes V, batteries E_j, observed rates r_j, link energies, weight κ
Ensure: parent function p

$A_j \leftarrow 0$, $p(j) \leftarrow \perp$ for all j; $\bar{r} \leftarrow$ mean of r_j
for $i \in V$ in increasing distance to the sink **do**
  if $p(i) \neq \perp$ **then**
    $u \leftarrow i$; **while** $u \neq s$: $A_u \leftarrow A_u + r_i$, $u \leftarrow p(u)$ ▷ book own traffic
    continue
  end if
  arc weights from (7); for j with $p(j) \neq \perp$ keep only $j \rightarrow p(j)$
  $P \leftarrow$ Dijkstra least-cost path from i to s
  for each u on P except s **do**
    if $p(u) = \perp$ **then** $p(u) \leftarrow$ successor of u on P
    end if
    $A_u \leftarrow A_u + r_i$
  end for
end for
return p

source's rate on every node of it. A node that already received a route as a relay keeps its next hop, and its own rate is booked along that fixed path when its turn comes. Rates r_i are the sink's count of packets received from i over the last 24 rounds, so no extra bits are needed. With $\kappa = 0$ the method is battery-weighted Dijkstra. Validation chose $\kappa = 0.5$, and Section 6.4 sweeps κ on fresh deployments. Algorithm 1 gives one round.

4.5. Ablations of booked-load Dijkstra

Load only removes the battery factor from (7). *Tiered load* replaces the sink-observed rates with load reported in four tiers (boundaries 0.15, 0.40, 0.75 of twice the largest historical rate, with a 0.05 hysteresis band). This is the Lightweight State Table design of our earlier work, and it is charged as in Section 3.3. In a centralised sink the observed rates are free, so the tiered variant cannot gain. It is reported to quantify what quantised sharing would cost in a setting where nodes, not the sink, need the load.

4.6. Upper reference: LP planner

The sink re-solves the LP (3)–(4) every round from the current batteries. Each node generates its observed mean rate, and a small total-energy term

Table 3: What each router needs ($n = 100$). Times are single-thread wall-clock per round, mean over the main bench.

Router	Information at the sink	Work per round	Extra bits	ms/round
Min-energy Dijkstra	positions	one Dijkstra	0	0.8
Battery-weighted Dijkstra	+ batteries	one Dijkstra	0	0.8
Load-balanced SPT [8]	positions, range R	rebuild on a death (max-flow)	0	0.5
Flow augmentation [1]	+ batteries, observed rates	$K = 30$ Dijkstra runs	0	17.2
Booked-load Dijkstra	+ batteries, observed rates	$\leq n$ Dijkstra runs	0	43.6
Booked-load, LEST 4-tier load	+ batteries, tiered load	$\leq n$ Dijkstra runs	8/packet, $2n$/round	43.4
LP planner (LP every round)	+ batteries, observed rates	one LP (HiGHS)	0	155.4

$\mu = 10^{-3}$ favours low-energy flows. Every node then sends to the next hop with the largest LP share; credits are reset at each solve, so this is a deterministic rounding of the flow, not proportional rotation. It uses the same information as the routers above and is the strongest realisable router we found. It needs an LP solver at the sink.

5. Evaluation protocol

Splits. Test topologies use seeds 0–29, each with all four scenarios (traffic seed = topology seed + 1000). Every free setting was chosen on separate validation topologies 500–511 (scenarios R, N, B) before any test run:

- flow augmentation: $x \in \{0.5, 1, 2, 5, 20, 50\}$, $H \in \{1, 2, 3, 6, 24, 96\}$, $K \in \{5, 10, 30\}$;
- Shan's range: $R \in \{90, 110, 125, 150, 200\}$ m;
- booked-load: $\kappa \in \{0.5, 1, 2\}$.

The robustness grid (Section 6.5) and the κ sweep (Section 6.4) use *fresh* topologies 2000–2029 that were never used for tuning or testing. Test runs were executed once with the settings fixed in the released code.

Statistical unit. A topology's primary score for a router is the mean of its R, N and B scores; D is reported separately. The four scenarios share a topology, so the 120 topology–scenario cases are not independent, and win/tie/loss counts over them are descriptive only. Paired comparisons use the 30 topology means. We lead with the mean paired difference and its 95% bootstrap confidence interval over topologies (20,000 resamples) [16], and the matched-pairs rank-biserial correlation r [17]. Two-sided Wilcoxon signed-rank p-values [18, 19] with Holm adjustment [20] over the eight comparisons of Table 5 are reported for completeness. The routers are deterministic, so a small p only states that one router is consistently ahead on this topology generator. The effect size and the interval carry the evidence.

6. Results

6.1. Main benchmark

Table 4 reports every router on the 30 test topologies.

- **Battery-weighted Dijkstra** reaches 85.2% of the bound on the R/N/B mean.
- **Within-round coordination** adds about 4.5 points. Flow augmentation in within-round increments reaches 89.9% and booked-load Dijkstra 89.7%; both beat battery Dijkstra on all 30 topologies.
- **The LP planner** reaches 96.6% with a worst topology of 93.9%.
- **Shan et al.'s tree and MTE** stay near 25%. In this radio model, a minimum-hop tree forces long, energy-expensive hops (d^4 beyond 87.7 m), and its balance is built once. MTE drains the same relays first.

Fig. 1 shows the topology-level pairs: booked-load Dijkstra is above battery Dijkstra everywhere and scattered on both sides of flow augmentation.

6.2. Paired comparisons

Table 5 gives the eight planned comparisons on topology means. The difference between booked-load Dijkstra and flow augmentation is -0.2 points with a 95% interval of $[-1.3, 0.8]$, so the two are indistinguishable. Over the 120 topology–scenario cases, which are descriptive only, booked-load Dijkstra wins 110, ties 7 and loses 3 against battery Dijkstra; flow augmentation wins 115, ties 4 and loses 1.

Table 4: Lifetime as a percentage of the fractional-flow bound (30 test topologies). R: daily surge; N: constant (MADII's static setting); B: bursts; D: pattern shift, not tuned on. S is the R/N/B mean; Worst is the lowest topology mean. The last column is the paired difference to battery-weighted Dijkstra on topology means, with a 95% bootstrap CI.

Router	R	N	B	S	Worst	D	vs battery Dijkstra
LP planner (LP every round)	96.2	98.3	95.2	96.6	93.9	95.7	+11.4 [+10.5, +12.3]
Flow augmentation [1]	88.3	92.4	89.0	89.9	82.9	88.9	+4.7 [+3.9, +5.5]
Booked-load Dijkstra	88.1	92.6	88.4	89.7	81.1	89.8	+4.5 [+3.6, +5.4]
Booked-load, LEST 4-tier load	88.2	92.3	87.9	89.4	81.7	88.8	+4.2 [+3.3, +5.2]
Battery-weighted Dijkstra	83.2	88.9	83.6	85.2	78.6	83.2	reference
Booked-load, load only (no battery)	41.3	39.7	41.2	40.7	26.6	40.9	−44.5 [−46.3, −42.8]
Min-energy Dijkstra	24.9	24.5	25.2	24.9	15.0	24.9	−60.4 [−62.0, −58.6]
Load-balanced SPT [8]	25.3	24.9	25.2	25.1	3.3	25.3	−60.1 [−63.3, −57.2]

Table 5: Planned paired comparisons on topology means ($n = 30$). Mean difference in points with a 95% bootstrap CI, topology wins/ties/losses, rank-biserial r, and Holm-adjusted Wilcoxon p over these eight comparisons.

Comparison	Difference [95% CI]	W/T/L	r	p_{Holm}
Booked-load Dijkstra vs Battery-weighted Dijkstra	+4.5 [+3.6, +5.4]	30/0/0	+1.00	1.5×10^{-8}
Flow augmentation [1] vs Battery-weighted Dijkstra	+4.7 [+3.9, +5.5]	30/0/0	+1.00	1.5×10^{-8}
Booked-load Dijkstra vs Flow augmentation [1]	−0.2 [−1.3, +0.8]	17/0/13	−0.05	0.839
LP planner (LP every round) vs Booked-load Dijkstra	+6.9 [+5.8, +8.1]	30/0/0	+1.00	1.5×10^{-8}
LP planner (LP every round) vs Flow augmentation [1]	+6.7 [+5.7, +7.6]	30/0/0	+1.00	1.5×10^{-8}
Booked-load, LEST 4-tier load vs Booked-load Dijkstra	−0.2 [−0.8, +0.3]	11/0/19	−0.29	0.354
Booked-load, load only (no battery) vs Booked-load Dijkstra	−49.0 [−51.0, −47.0]	0/0/30	−1.00	1.5×10^{-8}
Load-balanced SPT [8] vs Battery-weighted Dijkstra	−60.1 [−63.3, −57.2]	0/0/30	−1.00	1.5×10^{-8}

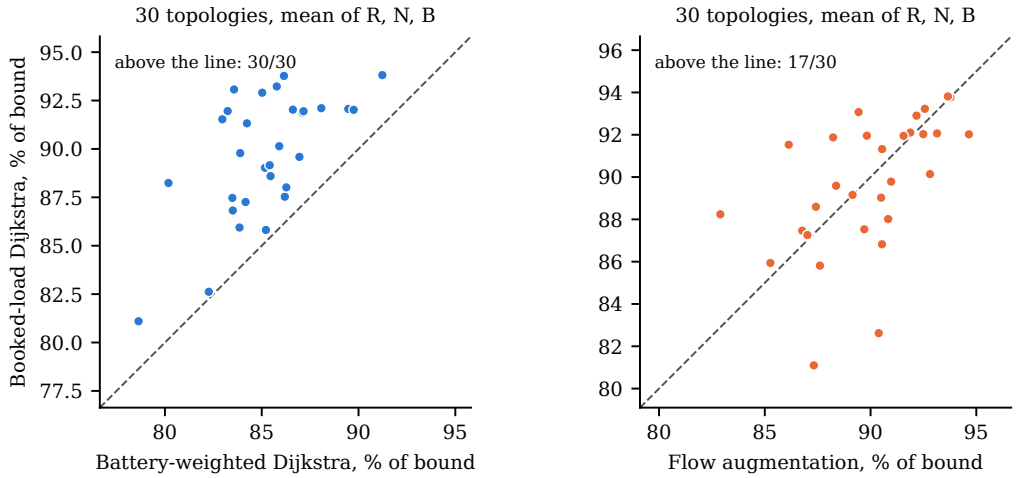


Figure 1: Topology means (R, N, B) of booked-load Dijkstra against battery-weighted Dijkstra (left) and against flow augmentation (right). Points above the dashed line favour booked-load Dijkstra.

6.3. How tight is the bound?

The bound is a relaxation, so a percentage of it is only meaningful if it is close to what trees can achieve. On every topology–scenario case, the best realised router gives a lower bound on the achievable lifetime. Averaged over the 120 cases that router reaches 96.4% of the bound, and its lowest case is 89.1%. The achievable optimum therefore lies within 3.6 points of the bound on average, and within 10.9 points in the worst case. A score of 85% therefore means at least $85/96.4 \approx 88\%$ of what the best known tree sequence achieved, and at most 85% of the true optimum.

6.4. Ablations and the load weight

Removing the battery term from booked-load Dijkstra lowers it to 40.7% (-49.0 points): load spreading alone keeps relaying through nodes that are running out. Reporting load in four quantised tiers instead of the sink's observed counts changes the score by -0.2 points (CI $[-0.8, 0.3]$). This is not distinguishable from zero, but the tiers buy nothing either, because a centralised sink already observes the traffic for free. On fresh topologies (Table 6), the gain over battery Dijkstra is positive for κ from 0.25 to 2, with a flat optimum for $\kappa \in [0.25, 1]$ (within one point of the best, $+5.6$ at $\kappa = 0.5$). Larger weights hurt: $\kappa = 4$ is 3.0 points *below* battery Dijkstra, plausibly

Table 6: Load weight κ of booked-load Dijkstra on 30 fresh topologies (seeds 2000–2029), R and N scenarios. $\kappa = 0$ is battery-weighted Dijkstra.

κ	R	N	Mean	vs $\kappa = 0$ [95% CI]
0	82.8	88.8	85.8	reference
0.25	89.3	93.2	91.3	+5.5 [+4.5, +6.5]
0.5	89.6	93.2	91.4	+5.6 [+4.6, +6.6]
1.0	89.0	92.6	90.8	+5.0 [+4.0, +6.1]
2.0	87.1	90.7	88.9	+3.1 [+2.0, +4.2]
4.0	80.4	85.3	82.8	−3.0 [−5.0, −1.0]

because an overweighted load term pushes traffic around busy relays onto weaker ones. The validation choice $\kappa = 0.5$ gives +5.6 [+4.6, +6.6] on these fresh topologies.

6.5. Robustness on fresh topologies

Table 7 and Fig. 2 repeat the comparison on fresh topologies under five changed conditions: smaller and larger networks in the same field, the sink at a corner, clustered deployments, and both changes together. Booked-load Dijkstra is ahead of battery Dijkstra by 0.4 to 44.3 points; the 95% interval is above zero in 4 of 5 conditions but includes zero for $n = 50$. The two coordination mechanisms are not interchangeable once the operating point changes. Flow augmentation is ahead for $n = 50$ (−3.6) and clustered layouts (−4.0); booked-load Dijkstra is ahead for $n = 200$ (+0.9), the corner sink (+6.3), clustered layouts with a corner sink (+25.5). Booked load gains most where traffic funnels hardest towards the sink, while flow augmentation does better in small or clustered networks. The LP planner is best wherever it was run except for clustered layouts with a corner sink, where booked-load Dijkstra reaches 59.8% against 49.8%. A plausible cause is that, with very long links, rounding the fractional flow to one next hop per node per round loses more than the planner gains. Absolute scores are also low there for every router (the best reaches about 60%), so the bound itself may be loose in that condition.

6.6. Lossy links and contention

Within-round coordination spreads traffic over more relays, which changes contention. We therefore probe the ideal-channel assumption once. Each link has a per-attempt reception rate that falls logistically with distance

Table 7: Robustness on 30 fresh topologies per condition (seeds 2000–2029), mean of R and N, % of the bound. The LP planner is omitted at $n = 200$ (its per-round solve made the full grid impractical; see Table 10). The last two columns are paired differences on topology means with 95% CIs.

Condition	MTE	Shan	Battery	FA	Booked	LP	Booked − Battery	Booked − FA
$n = 50$	30.0	19.0	87.3	91.3	87.7	97.2	+0.4 [−0.7, +1.6]	−3.6 [−5.0, −2.3]
$n = 200$	18.1	23.6	81.0	90.1	91.0	–	+10.0 [+8.8, +11.4]	+0.9 [+0.4, +1.6]
Sink at corner	30.5	27.7	69.2	79.3	85.6	86.8	+16.5 [+14.8, +18.2]	+6.3 [+5.5, +7.3]
Clustered	16.1	22.4	84.1	92.1	88.2	97.5	+4.0 [+2.0, +6.3]	−4.0 [−5.7, −2.4]
Clustered, sink at corner	6.8	3.1	15.5	34.3	59.8	49.8	+44.3 [+37.0, +51.1]	+25.5 [+19.7, +31.3]

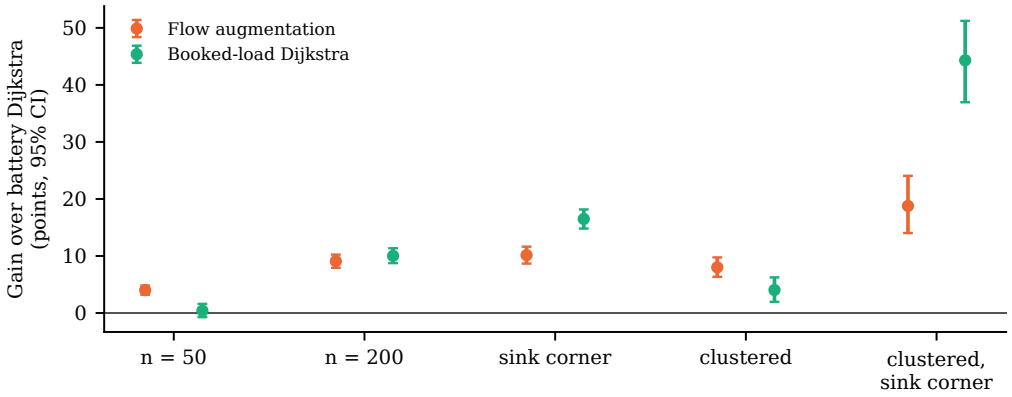


Figure 2: Gain over battery-weighted Dijkstra on fresh topologies, with 95% bootstrap intervals over topologies.

(50% at 120 m, width 15 m) under static log-normal shadowing ($\sigma = 0.1$), a common shape for the transitional region of low-power links [21]. ARQ allows up to four attempts, and both sender and receiver pay for every attempt. Routers and the bound use the expected energy per delivered bit, an ETX-style metric [22]. In the contention variant, each receiver's reception rate is further multiplied by $\exp(-I_j/C)$, where I_j is the number of bits sent within 100 m of the receiver in that round and $C = 2$ Mbit. No router knows this term. With lossy links and ARQ, booked-load Dijkstra and flow augmentation live 4.5 and 3.5 points longer than battery Dijkstra, and the LP planner 9.6 points longer, while delivery falls from 99.0% to 98.3%, 98.7% and 94.5%. With contention added, booked-load Dijkstra and flow augmentation live 4.6 and 3.4 points longer than battery Dijkstra, and the LP planner 11.5

Table 8: Lossy-link probe (test topologies 0–29, R and N). Lifetime as % of the loss-aware bound, paired difference to battery Dijkstra, delivery ratio over the lifetime, and payload delivered before the first death.

Link model	Router	Lifetime	vs battery [95% CI]	Delivery (%)	Mbit delivered
Lossy, ARQ	Min-energy Dijkstra	29.0	−54.9 [−56.9, −52.7]	99.4	14.2
Lossy, ARQ	Battery-weighted Dijkstra	83.9	reference	99.0	40.9
Lossy, ARQ	Flow augmentation [1]	87.4	+3.5 [+3.0, +4.1]	98.7	42.4
Lossy, ARQ	Booked-load Dijkstra	88.3	+4.5 [+3.7, +5.2]	98.3	42.7
Lossy, ARQ	LP planner (LP every round)	93.4	+9.6 [+8.7, +10.5]	94.5	43.4
Lossy + contention	Min-energy Dijkstra	24.4	−45.9 [−47.7, −44.0]	98.6	11.9
Lossy + contention	Battery-weighted Dijkstra	70.3	reference	97.7	33.8
Lossy + contention	Flow augmentation [1]	73.7	+3.4 [+2.6, +4.3]	97.2	35.2
Lossy + contention	Booked-load Dijkstra	74.9	+4.6 [+3.6, +5.6]	96.6	35.6
Lossy + contention	LP planner (LP every round)	81.8	+11.5 [+10.2, +12.9]	91.6	36.7

points longer, while delivery falls from 97.7% to 96.6%, 97.2% and 91.6%. The lifetime ranking of the ideal channel survives losses and contention, but every router that spreads traffic beyond battery Dijkstra's routes pays in delivery, and the more it gains in lifetime, the more it pays: the LP planner most of all. Under lossy links the lifetime gain is therefore a trade-off rather than a free improvement, and the payload delivered before the first death (last column) is the fairer single number.

6.7. Trace-driven traffic (Intel Lab)

Table 9 uses the Intel-derived traffic generator on 27 overlapping start windows. The windows share one layout and overlap in time, and some runs outlast the trace and are censored (scored on their observed rounds). We therefore report the results descriptively, as a second traffic generator with a different autocorrelation structure, and draw no inferential claim from them. At 1.0 J the mean scores are 96.0% for the LP planner, 90.6% for booked-load Dijkstra, 89.1% for flow augmentation and 85.2% for battery Dijkstra: the same three levels as on synthetic traffic. Booked-load Dijkstra is ahead of battery Dijkstra in 16 windows, tied in 11 (5 of them because both runs outlast the trace) and behind in 0.

6.8. Computation at the sink

Table 10 gives the single-thread time of one routing decision as the network grows at constant density, and the time of one sparse LP solve of the lifetime problem, which the planner repeats every round. One LP solve takes

Table 9: Trace-driven traffic from Intel Lab readings (27 overlapping windows, scaled 54-mote layout). Mean and worst % of the horizon-truncated bound; Cens. counts runs that outlast the trace. Descriptive only.

	$E_0 = 1.0$ J			$E_0 = 1.5$ J		
Router	Mean	Worst	Cens.	Mean	Worst	Cens.
LP planner (LP every round)	96.0	76.4	6	98.2	88.0	14
Flow augmentation [1]	89.1	61.2	5	94.2	73.1	13
Booked-load Dijkstra	90.6	68.9	5	95.0	80.6	13
Booked-load, LEST 4-tier load	86.6	61.2	5	94.3	70.2	13
Battery-weighted Dijkstra	85.2	60.6	5	91.1	60.7	13
Min-energy Dijkstra	35.3	24.0	0	42.3	25.3	0
Load-balanced SPT [8]	30.9	23.2	0	36.7	21.2	0

Table 10: Single-thread wall-clock time of one routing decision (ms) and of one sparse LP solve (s), constant node density.

n	Battery (ms)	FA (ms)	Booked (ms)	LP solve (s)
50	0.5	11.2	16.8	0.03
100	0.7	19.8	55.9	0.22
200	2.2	49.6	275.1	1.38
400	6.0	164.9	1698.9	29.04
800	22.5	621.2	11012.3	218.55

0.22 s at $n = 100$ but 219 s at $n = 800$, against 11.0 s for booked-load Dijkstra and 0.62 s for flow augmentation. Booked-load Dijkstra grows fastest among the solver-free routers (up to n Dijkstra runs per round, 18 times the time of flow augmentation at $n = 800$). With hour-long rounds a server-class sink can still afford the LP at these sizes, so in the modelled system avoiding the solver buys little. Solver-free coordination matters where the sink is an embedded gateway, the rounds are short, or the network is much larger, and in a distributed implementation, which this study does not evaluate.

7. Discussion

What the bound shows. The ranking is clear. Battery weighting alone moves MTE from 25% to 85% of the bound. Within-round coordination adds about 4.5 points, and per-round LP planning another 7. Since the best realised

router reaches 96.4% of the bound, the remaining distance to the bound is mostly what the solver buys, not slack in the relaxation.

When does avoiding the solver matter?. In the modelled system the sink is mains-powered and its computation is free (A2), and Table 10 shows that an LP solve fits within an hour-long round even at $n = 800$. Within this model the rational choice is therefore the LP planner, and we recommend it. Solver-free coordination is worth having only outside the model: on an embedded gateway without an LP solver, with short rounds, for much larger networks, or as the basis of a distributed implementation in which nodes compute routes themselves. We do not evaluate a distributed implementation. The coordinated routers are therefore offered as the best option *when the solver is not available*, not as an advance over it.

Two forms of within-round coordination. Both coordinated routers make later routing decisions of a round see the consequences of earlier ones. Flow augmentation does this through the residual energy predicted after each increment, booked-load Dijkstra through the traffic already booked on each relay. Both are successive-shortest-path constructions with incremental pricing, the idea of flow augmentation [1] applied within a round. They tie at the main operating point but diverge elsewhere (Table 7). A plausible explanation follows. Booked load prices congestion directly, in proportion to the traffic routed through a relay. That matters most where traffic funnels hardest, as with a corner sink, where all traffic converges on a few relays. Energy prediction reacts only through the energy those relays would spend within a short horizon, which is a small fraction of their battery. In small or clustered networks, where funnelling is weaker, the energy-based price is the better guide. The mechanism therefore matters, and the choice depends on the topology. Flow augmentation is the faster default (2.5 times faster at $n = 100$); booked-load Dijkstra is preferable where a single sink collects from a wide field. Neither works without the battery term.

Why Shan's tree and EBR-style designs score low here. Shan et al.'s construction is optimal for its own objective: a minimum-hop tree with balanced descendant counts under uniform energy. In a radio model whose transmit energy grows with d^2 to d^4 and with full power control (A4), the minimum-hop constraint forces long hops, and a tree built once cannot follow residual batteries. Its low score therefore reflects a mismatch between the protocol's

assumptions and this model, not a defect of the protocol. The same reasoning is why aggregation-based protocols are out of scope (Section 3.1).

Learned routing. Our corrected reconstruction of a recent multi-agent DRL router did not learn a non-trivial policy (Appendix Appendix B). The corrected reward prefers Dijkstra-like routes about eightfold, so the failure is one of learning, not of the objective. This says nothing definitive about MADII as published: a comparison with the authors' implementation is needed before any claim about learned routing in general.

7.1. Threats to validity

- **Analytical model.** Following MADII, the bench computes energy from radio equations per round and assumes ideal links and collision-free scheduling; it is not a packet-level simulation. The lossy-link probe stays within this analytical style (expected-value ARQ and a simple contention term). It shows that the lifetime gain survives but costs delivery. Validating the ranking in a packet-level simulator with a real MAC (e.g., IEEE 802.15.4 CSMA/CA) is the natural next step and could change the magnitudes.

- **Accounting.** Battery reports (16 bits) and the encoded advertisement are charged. Synchronisation, neighbour discovery and the reliability of the advertisement itself are not.

- **Traffic.** The synthetic generators are ours. The Intel-derived traffic is a trace-driven generator on a scaled layout, not measured radio traffic, and its overlapping, censored windows support no inferential claim.

- **Generality.** Fresh topologies cover $n \in \{50, 100, 200\}$, two sink positions and two layouts in one field size. Other densities, heterogeneous batteries and mobility are untested.

- **Baselines.** Flow augmentation is used as a per-round planner with tuned exponent and horizon. Shan's tree omits its distributed refinement step. The learned router is a reconstruction.

- **Reuse of test topologies.** The main test topologies 0–29 were also used by earlier versions of this study. Every setting was fixed on validation topologies, and the robustness grid and κ sweep use topologies never seen before.

8. Conclusion

Measured against the fractional-flow bound, multi-hop tree routing without aggregation falls into three levels:

- battery-weighted Dijkstra reaches about 85%;
- within-round coordination reaches about 90%, through flow augmentation run in increments or through booked-load Dijkstra, which tie at the main operating point;
- an LP planner re-solved every round reaches about 97%.

At the main operating point the best realised router comes within a few percent of the bound, so there the bound is a meaningful yardstick. Removing the battery term destroys either coordinated router, and tiered load reporting is unnecessary at a centralised sink. On fresh topologies the two coordination mechanisms diverge: booked load is clearly better where traffic funnels towards a corner sink, flow augmentation in small or clustered networks. A reconstruction of a recent learned router failed to learn a non-trivial policy even though its own reward strongly prefers one. We report that as a limit of our reconstruction, together with the corrections made to earlier versions of this study.

For practice, a sink that can afford an LP per round should usually use the planner; the one exception in our grid was a corner sink over a clustered field, where booked-load Dijkstra did better. A sink that cannot, because it is embedded or the network is large, should use within-round coordination: the 2004 flow augmentation by default, and booked-load Dijkstra when the sink sits at the edge of a large or clustered field. Future work should evaluate the authors' implementation of learned routers, distributed coordination with an explicit control protocol, and packet-level MAC simulation.

Appendix A. Corrections relative to earlier versions

Reviewing the code before this revision found the following errors. All results in this paper come from the corrected code, and the released repository records each fix with a regression test.

(a) **Booking omission.** In the sequential construction, a node that had already received its route as a relay on an earlier path never booked its

own traffic. Its own rate is now booked along its fixed path (Algorithm 1, lines 3–5). A test checks that booked loads equal the subtree sums of the final tree. With this fix the earlier "90.5%, 118 of 120 wins" became the smaller gain reported here.

(b) **Cumulative energy in the learned router's reward.** The energy term of MADII's reward (its Eq. 15 defines E_{start} at the start of the round) was computed since deployment. It is now per round.

(c) **Checkpoint selection on test topologies.** The learned router's checkpoints were selected on topologies 0–4, which belong to the test set. Selection now uses topologies 500–504.

(d) **Control accounting.** The route advertisement was charged 100 bits, which cannot encode n next-hop identifiers. Residual-battery reports were assumed free, while load reports were charged. Both are now charged as in Section 3.3, identically for every router.

(e) **Metrics.** The hop count included one extra hop, and the tiered-load header was counted as delivered data. Both are fixed.

(f) **Statistical unit.** Earlier tests treated 120 topology–scenario cases as independent. They now use 30 topology means.

Appendix B. Learned-router reconstruction: a reproducibility failure

No code for MADII [2] is available. Our reconstruction uses an Informer-style encoder ($d_{\text{model}} = 128$, two layers, four heads) with a dot-product next-hop head, not the full published encoder–decoder. It is trained for 1,000 episodes: 300 with replay transitions from FCM, HBA and POA experts, then 700 with ε decaying from 0.1 to 0.01. It uses Adam (10^{-4}), batch 64, replay 10,000, discount 0.99, Double DQN [23], and checkpoint selection on validation topologies only. Three training seeds were run. With the corrected reward, the validation-selected checkpoints of all three seeds reach 8.27 rounds to first death (4.3% of the fractional bound, legacy accounting). This equals direct transmission to the sink (8.27 rounds): in the first round of every test topology the selected models send every source straight to the sink. This is not what the reward asks for. Scored with the same corrected reward (MADII's Eq. 8), battery-weighted Dijkstra earns 8.1 times more reward

than direct transmission in the first round, and 15.3 times more discounted return ($\gamma = 0.99$) over an episode. The collapse is therefore a learning failure of the reconstruction (plausibly credit assignment across 100 agents sharing one round return), not a property of the reward.

This outcome characterises our reconstruction, not MADII: the published paper reports 19 rounds to first death in its own experiments. We therefore do not include the reconstruction in the main comparison.

CRediT authorship contribution statement

Shubham Mittal: [roles]. **Aditya Periwal:** [roles].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The simulator, every router, all raw result files, the corrected learned-router checkpoints and the scripts that regenerate every table and figure are available at `https://github.com/ShubhamMittal-17/madii-repro` (branch `aditya/wsn-routing`, directory `revision/`). The Intel Berkeley Lab data are publicly available [15].

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) and ChatGPT/Codex (OpenAI) to assist with code development, code review and debugging, experiment automation, statistical analysis and manuscript drafting and revision. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Acknowledgements

[Funding statement, or: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.]

References

[1] J.-H. Chang, L. Tassiulas, Maximum lifetime routing in wireless sensor networks, IEEE/ACM Transactions on Networking 12 (4) (2004) 609–619.

[2] J. Yang, W. Li, C. Li, L. Zhang, L. Liu, An energy-efficient and transmission-efficient adaptive routing algorithm using deep reinforcement learning for wireless sensor networks, IEEE Internet of Things Journal 12 (23) (2025) 50414–50426. doi:10.1109/JIOT.2025.3609624.

[3] M. U. Younus, M. K. Khan, M. R. Anjum, S. Afridi, Z. A. Arain, A. A. Jamali, Optimizing the lifetime of software defined wireless sensor network via reinforcement learning, IEEE Access 9 (2021). doi:10.1109/ACCESS.2020.3046693.

[4] W. R. Heinzelman, A. Chandrakasan, H. Balakrishnan, Energy-efficient communication protocol for wireless microsensor networks, in: Proc. 33rd Hawaii International Conference on System Sciences, 2000.

[5] E. W. Dijkstra, A note on two problems in connexion with graphs, Numerische Mathematik 1 (1959) 269–271.

[6] R. Duan, J. Mao, X. Mao, X. Shu, L. Yin, Breaking the sorting barrier for directed single-source shortest paths, in: Proc. 57th Annual ACM Symposium on Theory of Computing (STOC), 2025.

[7] H. Dai, R. Han, A node-centric load balancing algorithm for wireless sensor networks, in: Proc. IEEE Global Telecommunications Conference (GLOBECOM), 2003.

[8] F. Shan, W. Liang, J. Luo, X. Shen, Network lifetime maximization for time-sensitive data gathering in wireless sensor networks, Computer Networks 57 (2013) 1063–1077. doi:10.1016/j.comnet.2012.12.005.

[9] O. A. Mahdi, Y. R. B. Al-Mayouf, A. B. Ghazi, M. A. Mohammed, A. W. A. Wahab, M. Y. I. B. Idris, An energy-aware and load-balancing routing scheme for wireless sensor networks, Indonesian Journal of Electrical Engineering and Computer Science 12 (3) (2018) 1312–1319. doi:10.11591/ijeecs.v12.i3.pp1312-1319.

[10] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: Proc. AAAI Conference on Artificial Intelligence, 2021.

[11] Q. Tang, W. Sun, Z. Liu, Q. Li, X. Yuan, Multi-agent reinforcement learning based dynamic self-coordinated topology optimization for wireless mesh networks, Journal of Network and Computer Applications 239 (2025) 104177.

[12] F. Zhou, A. Lischka, B. Kulcsar, J. Wu, M. H. Chehreghani, G. Laporte, Learning for routing: A guided review of recent developments and future directions, arXiv:2507.00218 (2025).

[13] N. Krishna, G. N. Sundar, D. Narmadha, J. Gowda V, Energy efficient data transmission in WSN for network lifetime enhancement, in: Proc. 7th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI), IEEE, 2026, pp. 183–189.

[14] K. Jerabek, J. Luxemburk, R. Plny, J. Koumar, J. Pesek, K. Hynek, When simple model just works: Is network traffic classification in crisis?, arXiv:2506.08655 (2025).

[15] Intel Berkeley Research Lab, Intel lab data, `http://db.csail.mit.edu/labdata/labdata.html` (2004).

[16] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[17] D. S. Kerby, The simple difference formula: An approach to teaching nonparametric correlation, Comprehensive Psychology 3 (2014).

[18] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[19] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine Learning Research 7 (2006) 1–30.

[20] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[21] M. Zuniga, B. Krishnamachari, Analyzing the transitional region in low power wireless links, in: Proc. IEEE SECON, 2004.

[22] D. S. J. De Couto, D. Aguayo, J. Bicket, R. Morris, A high-throughput path metric for multi-hop wireless routing, in: Proc. 9th Annual International Conference on Mobile Computing and Networking (MobiCom), 2003.

[23] H. van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, in: Proc. AAAI Conference on Artificial Intelligence, 2016.